

EE533_Lab8

Github repo: <https://github.com/hzhang2422/EE-533-Team4/tree/main/Lab8>

1. Convertible FIFO Design

Interface with NetFPGA and Process:

```
// NetFPGA interface
input [63:0] in_data,
input [7:0] in_ctrl,
input in_wr,
output reg in_rdy,
output reg [63:0] out_data,
output reg [7:0] out_ctrl,
output reg out_wr,
input out_rdy,

// Processor interface
input [63:0] proc_data_in,
input [7:0] proc_addr_in,
output reg [63:0] proc_data_out,
input proc_web,
input proc_memEn,
input pin_di, //processor data process request
output reg p_en, //processor enable
```

Pin_di is a input signal sent by the processor indicating that processor is ready to process data.

FSM:

We define 4 stages

```
// FSM state definitions
parameter IDLE = 4'b0001,
          RX   = 4'b0010,
          // PROCESS = 4'b0100,
          EX   = 4'b0100,
          TX   = 4'b1000;
```

1. IDLE stage

```
IDLE: begin
    in_rdy = 1'b1;
    if (in_wr_reg) begin
        next_state = RX;
        web = 1'b1;
    end else next_state = IDLE;
end

IDLE: if (in_wr_reg) begin
    count <= count + 1;
    wp <= wp + 1;
    packet_count <= packet_count + 1;
end
```

In the IDLE stage, pulling up the output signal for NetFPGA, and waiting for the write enable signal to receive data from NetFPGA datapath. Also, increasing write pointer of FIFO and packet_count signal.

2. RX stage

```
RX: begin
    in_rdy = ~control_nonzero;
    if (in_wr_reg) begin
        web = 1'b1;
        next_state = control_nonzero ? RXDONE : RX; //if 1, end of the packet
    end else next_state = RX;
end
RX: if (in_wr_reg) begin
    count <= count + 1;
    wp <= wp + 1;
    packet_count <= packet_count + 1;
end
```

In RX stage, it is basically the same as IDLE stage, except for resetting in_rdy signal based on in_ctrl data. If control_nonzero signal is 1, it means the end of the packet. The state machine will switch to the next stage, otherwise keeping stay on RX stage until receiving the last packet.

3. RXDONE stage

```
RXDONE: begin
    if(in_wr_reg) begin
        web = 1'b1;
        next_state = EX;
    end else next_state = RXDONE;
end
RXDONE: if (in_wr_reg) begin
    count <= count + 1;
    wp <= wp + 1;
    packet_count <= packet_count + 1;
end
```

This stage is used to make sure that the last word of the packet is received.

4. EX stage

```
EX: begin
    p_en = ~p_di;
    addrb = proc_addr_in;
    enb = proc_memEN;
    web = proc_web;
    dinb = {8'b0, proc_data_in};
    proc_data_out = doutb[63:0];
    next_state = p_di ? TX : EX;
end
```

In EX stage, packet can be processed by our processor. Processor can read packet from the SRAM, and write back to the SRAM after processing. When p_di is set to 1, which means the processing is done, it will switch to TX stage.

5. TX stage

```

TX: begin
    addrb = rp; //read pointer as address
    enb = 1'b1; //enb may change by proc_memEN in EX stage
    if (out_rdy) begin
        out_wr_w = 1'b1;
        next_state = (count == 8'd1) ? IDLE : TX;
    end else next_state = TX;
end

TX: if (out_rdy) begin
    rp <= rp + 1;
    count <= count - 1;
end

```

In TX stage, we send the processed data to NetFPGA from the SRAM by moving the read pointer as the read address.

Dual-port SRAM module

```

// Memory block instance
Memblock fifo1(
    .addra(memAddressQuery),
    .addrb(addrb),
    .clka(clk),
    .clkb(clk),
    .dina(72'b0),
    .dinb(dinb),
    .douta(memDataOut),
    .doutb(doutb),
    .enb(enb),
    .wea(1'b0),
    .web(web)
);

```

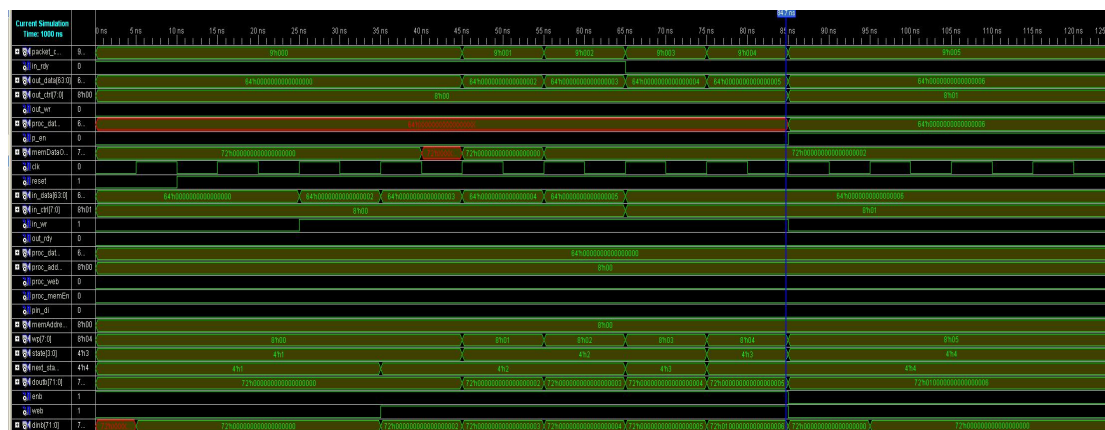
2. Simulation of FIFO Module

1. Send packet from NetFPGA to FIFO

```
task send_packet;
begin
    input [4:0] packet_size;
    input [63:0] base_packet;
    integer i;
    wait(in_rdy); //wait for IDLE
    //send the first few packets
    for(i=0; i<packet_size-1; i=i+1)
    begin
        @(posedge clk);
        in_wr = 1;
        in_ctrl = 8'h00; //good packet
        in_data = base_packet + i;
        //
        @(posedge clk);
        while (!in_rdy) begin
            @(posedge clk);
        end
    end

    //send the last packet
    @(posedge clk);
    in_ctrl = 8'h01;
    in_data = base_packet + packet_size - 1;
    in_wr = 1;
    @(posedge clk);
    @(posedge clk);
    in_wr = 0;

    $display("Send packet with %0d words at %t", packet_size, $time);
end
endtask
```



2. Modify packet by processor

```

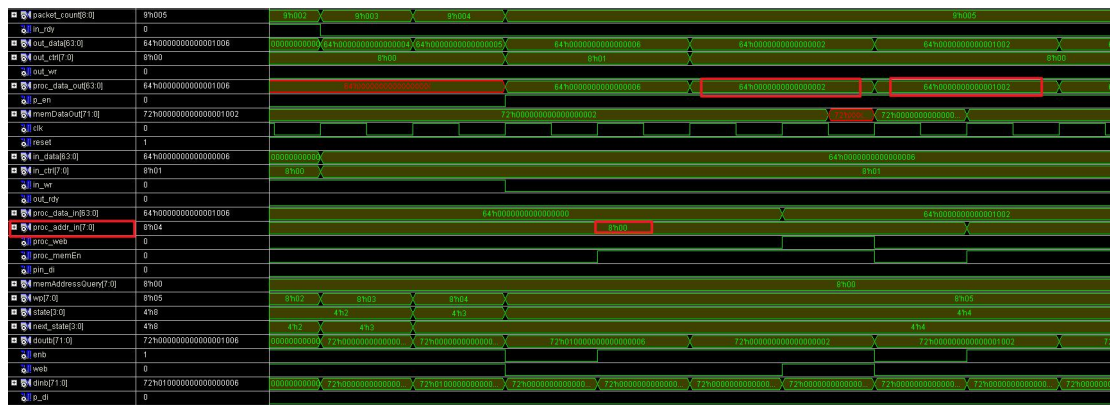
task process_packet;
    input [7:0] start_address;
    input [4:0] packet_size;
    integer i;
    begin
        wait(p_en); //wait for EX stage
        for(i=0; i<packet_size; i=i+1)
            begin
                @(posedge clk);
                proc_addr_in = start_address+i;
                proc_memEn = 1'b1; //read data from sram
                proc_web = 1'b0;

                @(posedge clk); // read data occurs in next clk

                //modify data then write back to sram
                @(posedge clk);
                proc_data_in = proc_data_out + 64'h0000_0000_0000_1000;
                proc_web = 1'b1;

                @(posedge clk);
                proc_memEn = 1'b0;
                proc_web = 1'b0;
            end
        end
    end
endtask

```



We can see from the figure above, the packet from the same address is different after processing.

3. Recieve processed data

```

task recieve_packet;
input [4:0] packet_size; //the number of reception data
integer i;
begin
    out_rdy = 1'b1;

    for(i=0; i<packet_size; i=i+1)
    begin
        wait(out_wr);
        @(posedge clk);
    end

    out_rdy = 1'b0;
end
endtask

```

